

Introduction

Notre projet d'application pour analyser les sentiments sur twitter est disponible sur git (<https://gitlab.univ-lille.fr/samuel.dubuisson.etu/PJE>). Pour le lancer, il suffit de se placer à la racine du projet et de lancer la commande `make compile ; make run`.

Description générale du projet

Problématique

L'objectif de ce projet est le développement d'une application permettant d'utiliser plusieurs approches afin de classifier le sentiment général de tweets. Pour cela, nous implémentons plusieurs algorithmes de classification. Nous allons nous demander quelle méthode de classification est la plus efficace pour analyser le sentiment d'un tweet, et si cela varie selon la taille de la base de données.

L'architecture

Notre projet est composé de trois parties principales, dans l'objectif de séparer les dépendances de manière judicieuse :

1) La composante algorithmique

Nous avons les différents algorithmes de classification héritant d'une classe abstraite `Algorithme`. Plusieurs classes sont également utilisées pour faciliter l'implémentation des variantes des algorithmes de Bayes.

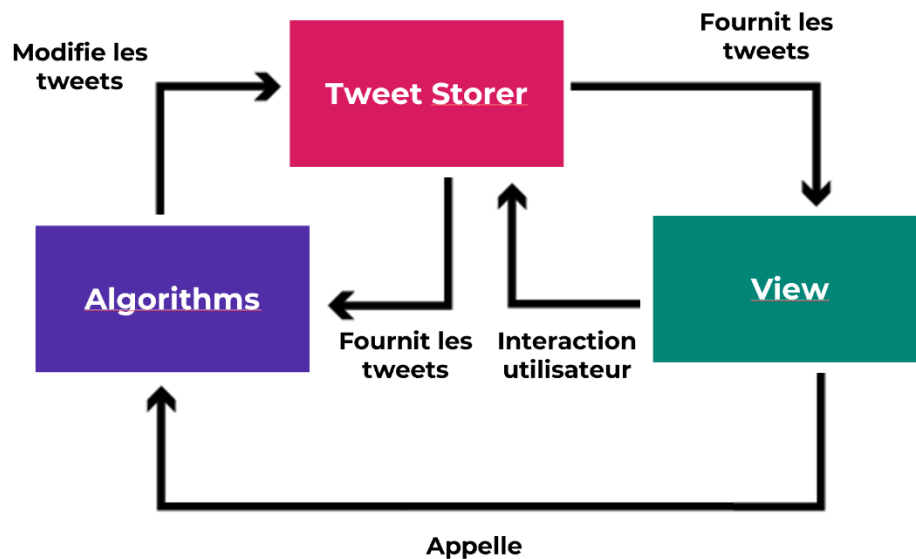
2) La composante graphique

Elle a été réalisée avec `tkinter`. On dispose d'une classe principale, et de classes qui gèrent les composantes annexes (boutons, zones de texte et d'affiche, menus déroulants pour sélectionner les options des algorithmes, etc.)

3) La composante de liaison

Cette partie du projet a la charge de la communication entre l'affichage et le modèle de calcul. Elle est composée du `TweetStorer`, qui se charge de stocker les tweets de test, d'apprentissage, l'analyse en cours, et de charger les données depuis un document `csv`.

Un résumé en image de notre architecture :



On a également une partie en charge de l'expérimentation (fichier d'expérimentation, Kfold). Des tests unitaires ont également été réalisés pour certains points du projet.

L'organisation au sein du binôme

Nous avons divisé le travail en 7 parties :

- L'interface graphique
- Traitement des données
- L'algorithme Dictionnaire
- L'algorithme KNN
- L'algorithme Bayes
- L'analyse expérimentale
- Le rapport/Powerpoint

Nous avons quasiment tous les deux *commité* sur l'ensemble de ces parties tout en nous spécialisant petit à petit : Yanis s'est principalement occupé de l'interface, de l'algorithme Dictionnaire, du traitement des données et du PowerPoint pendant que Samuel travaillait prioritairement sur KNN, Bayes, l'analyse expérimentale et le rapport.

Détails des différents travaux réalisés

Préparation/nettoyage des données

Les données sont récupérées depuis fichier csv. Les contenus des tweets sont ensuite nettoyés selon plusieurs règles qui facilitent l'analyse du tweet : suppression des tags, des signes de ponctuation, des nombres, etc. Chaque tweet a un champ qui fait référence au contenu de base (pour l'affichage) et au contenu nettoyé pour les algorithmes.

Bases d'apprentissage construites

2 bases d'apprentissages sont utilisées :

- Test data manual (500 tweets)
- Covid-19 database (14 000 tweets)

Les tweets de la base covid-19 sont disponibles dans leur intégralité, mais au vu de la taille de la base, nous avons également créé des sous-ensembles de ce dataset de tailles suivantes : 1 000 tweets, 5 000 tweets, 10 000 tweets

On dispose également d'une base de tweets Avengers, mais les tweets ne sont pas étiquetés. Elle est donc utilisable pour lancer l'application et déterminer les sentiments des tweets, mais pas en tant que base d'apprentissage ou pour la phase d'expérimentation (rôle de la base COVID et test manual).

Algorithmes de classification

Les trois modèles de classification sont disponibles : Dictionnaire, KNN, et Bayes.

Dictionnaire

Cet algorithme utilise deux fichiers qui contiennent chacun une base de mots positifs et négatifs. Il calcule simplement le nombre d'occurrences de mots positifs et négatifs dans un tweet, et classe le sentiment du tweet en fonction de ce résultat. Une base de mots positifs et négatifs avait été fournie, et nous avons rajouté d'autres pour le rendre plus performant.

KNN

Cet algorithme regarde pour un tweet les tweets de la base d'apprentissage les plus "proches", et détermine en fonction de cela la classe du tweet concerné. Nous avons implémenté plusieurs variantes de cet algorithme, en fonction de plusieurs paramètres :

- Le nombre de voisins (K) : l'algorithme va prendre en compte seulement les K plus "proches" voisins
- La méthode de distance : permet de calculer à quel point un tweet est proche d'un autre. Les

distances implémentées ont été la distance classique, et la distance de jaccard.

- La fonction de pondération : à quel point un tweet va être pris en compte en fonction de sa proximité avec un autre tweet ? Par défaut, tout tweet compris dans les k voisins a la même importance que les autres. Mais on peut décider que son importance est inversement proportionnelle à la distance, ou à la distance au carré (deux variantes proposées dans l'interface).

Bayes

C'est un algorithme probabiliste qui va utiliser une base d'apprentissage pour classer nos nouveaux tweets, en calculant les probabilités conditionnelles grâce à la formule de Bayes. Pour chaque mot ou groupe de mot d'un tweet, les probabilités d'apparition selon les différentes classes seront calculées, puis assemblées, pour déterminer la classe la plus probable.

Tout comme KNN, diverses variantes ont été implémentées sous forme de paramètres et font varier le comportement de l'algorithme :

- Type de représentation :
 - Par présence : on ne prend pas en compte le nombre d'apparitions d'un mot. Une seule apparition équivaut à cent apparitions.
 - Par fréquence : le nombre d'apparitions de chaque mot est pris en compte.
- Les ensembles de mots pris en compte :
 - Uni-gram : prend en compte les mots un par un
 - Bi-gram : prend en compte les groupes de 2 mots consécutifs (en fenêtre glissante)
 - Uni-gram et Bi-gram : combinaison des deux

Interface graphique

L'interface à l'ouverture du projet :

The screenshot shows the 'Twitter Sentiments Analysis App' interface. It features a control panel at the top with a dropdown menu for 'Algorithme Dictionnaire', buttons for 'Dictionnaire', 'Importer fichier d'apprentissage', 'Importer fichier de test', 'Analyser les tweets', and 'Editer'. Below this is a text area containing a list of tweets, each preceded by a classification label and a prediction, such as 'T: 0 A: -1 -> LeBron James got in a car accident...'. Annotations with arrows point to various parts of the interface: an orange arrow points to the 'Algorithme Dictionnaire' dropdown with the text 'Sélectionner l'algo de classification'; a green arrow points to the 'Analyser les tweets' button with the text 'Lancer l'algo de classification'; a red arrow points to the 'Importer fichier de test' button with the text 'choisir un nouveau fichier d'apprentissage puis notre arborescence'; a blue arrow points to the 'Editer' button with the text 'éditer un tweet et sa classe à la main'; and a purple arrow points to the tweet list with the text 'Zone d'affichage des tweets: T : <label> A : <prédiction> -> tweet content (T pour Tweet et A pour Analyse)'. The interface also includes a small input field with '-1' and a list of tweets at the bottom.

Cliquer sur le bouton “Analyser les tweets” lancera donc l’algorithme de classification de l’algorithme choisi sur le fichier de test sélectionné, en apprenant du fichier d’apprentissage choisi. Après les quelques secondes d’exécution de l’algorithme, les résultats sont actualisés et affichés dans la zone d’affichage des tweets.

Pour certains algorithmes (KNN et Bayes), des paramètres sont sélectionnables et modifiables, voici comment l’interface se présente dans ce cas :

Les différents paramètres sont ici modifiables grâce aux menus déroulants sous la zone d’affichage.

Résultats de la classification avec les différentes méthodes et analyse

Mise en place de l'expérimentation

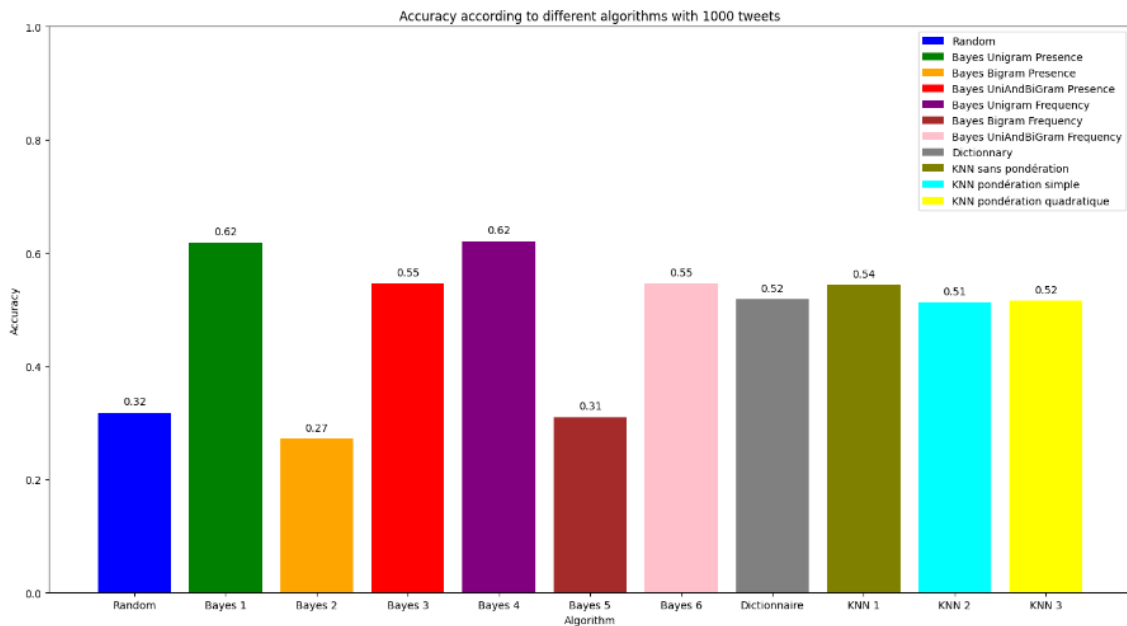
Nous avons mis en place un système de cross-validation pour l’expérience, où le dataset sélectionné est séparé en 10 parties égales. On utilise ces 10 parties pour réaliser 10 phases d’expérimentation, où l’ensemble de test/validation et l’ensemble d’entraînement alternent. Le score de précision est obtenu en faisant la moyenne de la précision à chaque étape.

- Pour cette expérience, nos variables indépendantes sont :
 - L'ensemble d'apprentissage utilisé
 - Le nombre de tweets étudiés
 - L'algorithme utilisé (Random, Dictionnaire, KNN, Bayes)

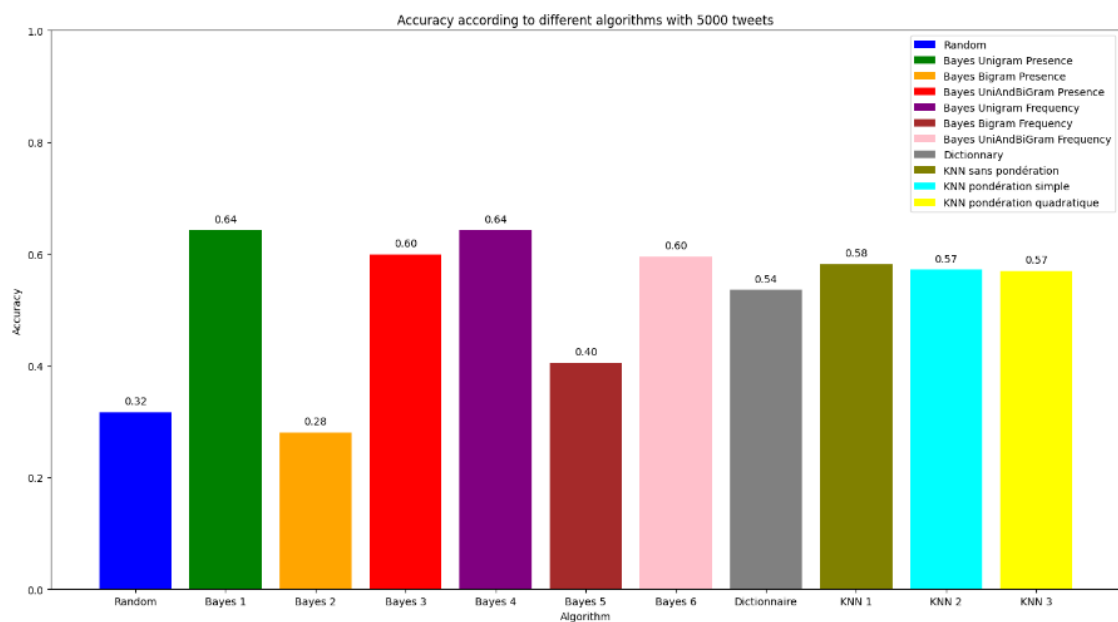
- Les paramètres (la fonction de pondération utilisée pour KNN, le nombre de mots pris en compte et la méthode de représentation pour Bayes).
- Les variables dépendantes seront donc les scores de précision obtenus.
- L'algorithme de référence sera l'algorithme Random.

Résultats avec les bases du COVID

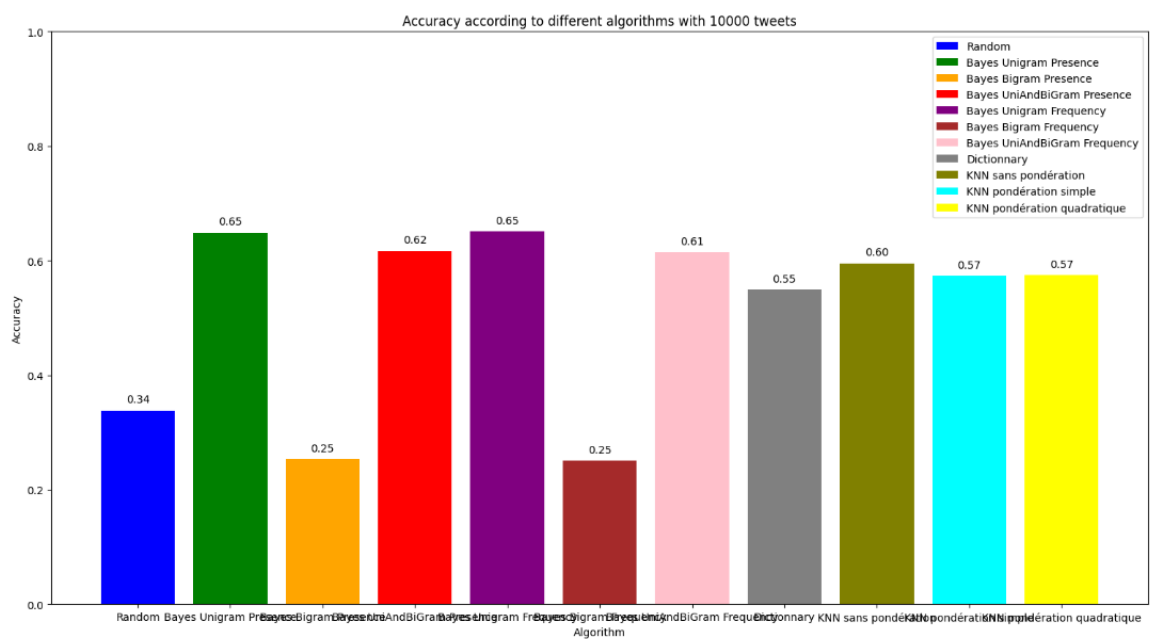
- Un échantillon de 1000 tweets :



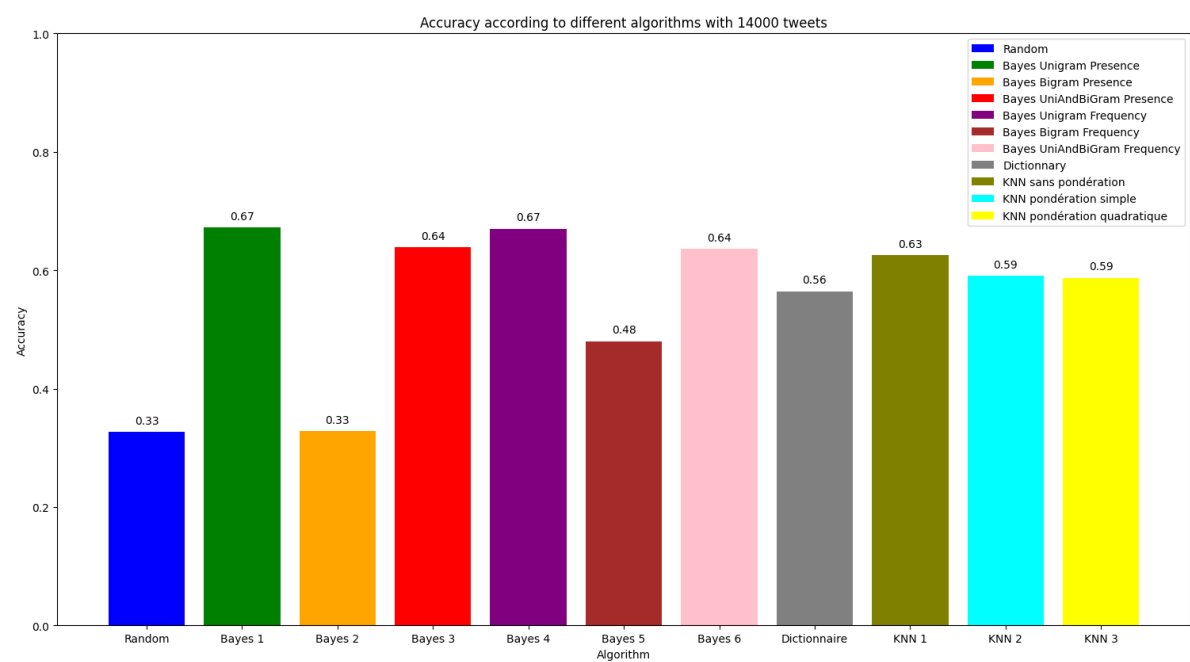
- Un échantillon de 5000 tweets :



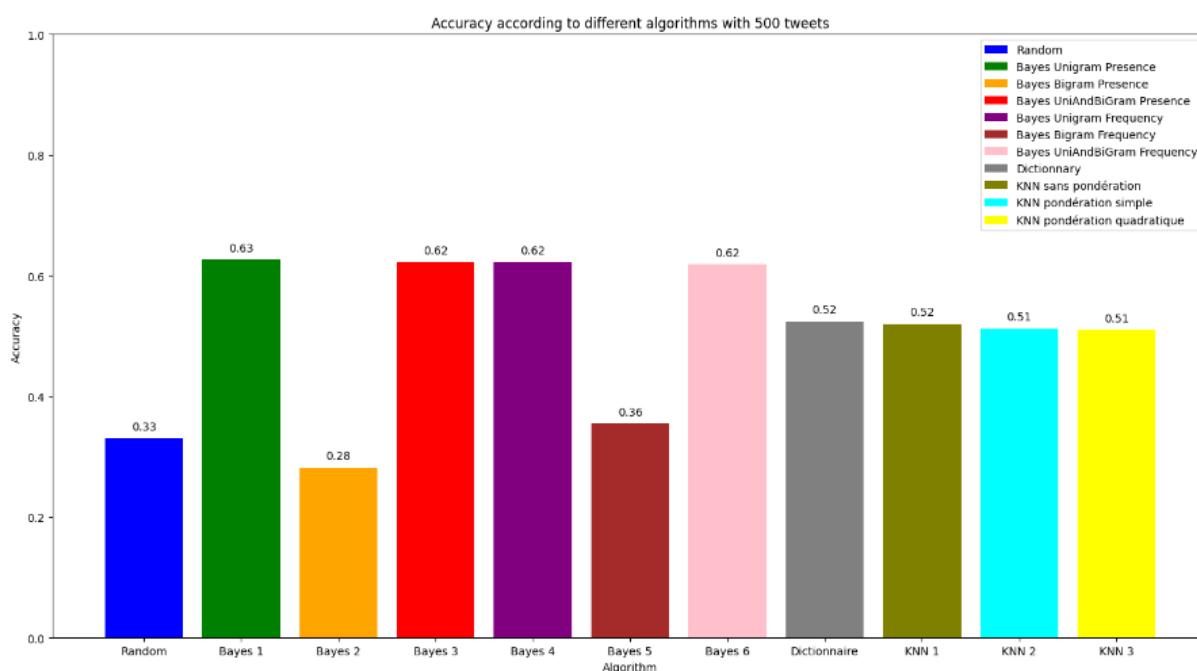
- Un échantillon de 10 000 tweets :



- Échantillon total de 14 000 tweets :



Résultat avec la base test_data_manual (500 tweets)



Interprétation des résultats

Algorithme Random

Comme imaginé, Random a un score de précision de 33% ($1 / \text{le nombre de classes} = 1/3$)

Algorithme Dictionnaire

Dictionnaire a un score de précision plutôt constant, peu importe la base et le nombre de tweets. Il se situe un peu au-dessus de 50% (entre 52% et 56% selon les expériences). Cela est logique, car il n'utilise pas de base d'apprentissage : il n'utilise que sa base de données de mots négatifs et positifs. Plus ces mots seront nombreux, et plus son score sera précis. Il ne pourra cependant jamais dépasser un certain stade de précision, car il manque de compréhension sur la phrase.

Algorithme KNN

Le score de précision de KNN dépend déjà davantage de la base de données utilisée. Il se situe entre 51% et 63%. Il est donc légèrement plus performant que l'algorithme Dictionnaire. Parmi les différentes variantes de KNN utilisées pour l'expérimentation, la plus performante est celle qui utilise une fonction de pondération constante, c'est-à-dire où la proximité d'un voisin par rapport à un tweet concerné n'est pas prise en compte. On remarque que l'efficacité de KNN est proportionnelle au nombre de tweets dans la base. Cela semble logique, car avoir plus de tweets permettra d'avoir plus de voisins auxquels se comparer et permettra une meilleure précision.

Algorithme Bayes

Bayes est l'algorithme qui est le plus performant. Les résultats obtenus sont variés selon les paramètres utilisés, et de la taille des échantillons, donc détaillons-les :

- Unigram et présence : **62% à 65%**
- Bigram et présence : 27% à 33%
- Uni/Bi gram et présence : 55% à 64%
- Unigram et fréquence : **62 à 67%**
- Bigram et fréquence : 31% à 48%
- Uni/Bi gram et fréquence : 55% à 64%

Dans la majorité des cas, les scores de précision les plus hauts sont obtenus sur la base COVID avec 14 000 tweets, et les scores les plus faibles avec la base COVID 1000 tweets ou Test-manual 500 tweets.

À travers ces résultats, on observe donc les performances suivantes sur les paramètres utilisés :

- Fréquence est légèrement plus performant que Présence (de l'ordre de quelques points)
- Unigram est plus performant qu'Unigram qui est lui-même plus performant que Bi-gram.
 - Cela est selon nous explicable par le fait que pour être performant et mesurer efficacement l'apparition de groupement de mots, bigram a besoin d'un nombre très important de tweets.
 - Bigram est en effet intéressant pour reconnaître des expressions clés de deux mots, comme "chef d'oeuvre" par exemple, mais nous ne sommes même pas sûrs de rencontrer de telles expressions dans une base de données avec peu de tweets, et quand on compare au nombre de combinaison de deux mots successives, il y a peu de chances que ces informations soient mises en valeur.
 - On remarque cependant qu'avec la base de données la plus fournie (COVID - 14 000 tweets), la précision du modèle bigram commence à devenir intéressante (bien qu'inférieure à unigram et unigram, elle s'approche des 50%). Il pourrait être pertinent d'analyser cet algorithme avec une base de données comprenant encore plus de tweets.

Conclusion

Plusieurs modèles de classification ont été implémentés : Dictionnaire, KNN, Bayes avec des variantes pour la majorité d'entre eux. Dictionnaire est en moyenne le moins efficace d'entre eux, il est cependant plus sûr quand on dispose de peu de tweets dans notre base d'apprentissage car il ne l'utilise pas, ayant son propre registre de mots. KNN et Bayes sont tous les deux influencés par le nombre de tweets dont on dispose dans notre base d'apprentissage, et sont plus efficaces quand la taille de la base s'agrandit. Les meilleures variantes de Bayes sont en moyenne plus efficaces que KNN : les plus efficaces d'entre elles sont Unigram + Présence et Unigram + Fréquence sont les plus efficaces.